

Plano de Desenvolvimento Completo — TuringOS

1. Visão geral

O TuringOS é um mini sistema operacional educacional inspirado em Unix, desenvolvido em C e Assembly x86, com foco em Interface Hardware/Software. O projeto tem como objetivo demonstrar, na prática, o funcionamento básico de um sistema operacional: processo de boot, entrada em Assembly, kernel em C, acesso direto à memória de vídeo, organização modular de drivers, terminal, shell, teclado, interrupções, depuração e engenharia reversa de binários.

O desenvolvimento será feito de forma incremental, com versões pequenas e funcionais. Cada etapa deve produzir uma melhoria visível ou verificável no sistema, evitando que o projeto fique grande demais sem entregar funcionalidades testáveis.

2. Estado atual do projeto

Versão atual: v0.1 — Kernel mínimo inicializável

A versão inicial do TuringOS já foi implementada e validada no QEMU.

Funcionalidades já concluídas:

- Estrutura inicial do projeto;
- Boot via GRUB;
- Cabeçalho Multiboot em Assembly;
- Entrada inicial em `boot.asm`;
- Configuração manual de pilha;
- Chamada da função `kernel_main`;
- Kernel mínimo em C;
- Escrita direta na memória VGA em `0xB8000`;
- Script de linkedição `linker.ld`;
- Configuração do GRUB;
- Makefile para build, ISO e execução;
- Execução bem-sucedida no QEMU.

Fluxo atual:

```
QEMU
↓
GRUB
↓
boot/boot.asm
↓
kernel_main()
```

↓
VGA text mode

Mensagem exibida atualmente:

TuringOS v0.1 - Kernel loaded successfully.

Essa versão prova que o kernel é carregável, executável e capaz de interagir diretamente com hardware virtualizado.

3. Estratégia geral de desenvolvimento

O projeto será desenvolvido por versões, usando branches específicas para cada funcionalidade.

Fluxo de trabalho recomendado:

```
main
↓
feature/nome-da-funcionalidade
↓
commit
↓
push
↓
pull request
↓
merge na main
```

Regras gerais:

- A branch `main` deve conter apenas versões estáveis;
- Cada etapa deve ser feita em uma branch própria;
- Cada branch deve ter um objetivo claro;
- Cada funcionalidade deve ser testada no QEMU antes do merge;
- Arquivos gerados, como `build/`, `.o`, `.bin` e `.iso`, não devem ser versionados;
- A documentação deve ser atualizada junto com o código;
- Cada etapa deve gerar evidências: prints, logs, comandos executados ou descrição técnica.

4. Roadmap geral

Versão	Nome	Objetivo	Status
v0.1	Kernel mínimo	Inicializar kernel via GRUB e escrever na VGA	Concluído
v0.2	Driver VGA e terminal	Organizar saída de vídeo em módulos	Próximo

Versão	Nome	Objetivo	Status
v0.3	Interface textual inicial	Criar tela inicial e terminal mais apresentável	Planejado
v0.4	Shell básico	Criar comandos internos simples	Planejado
v0.5	Teclado PS/2	Ler entrada do teclado	Planejado
v0.6	Interrupções	Criar IDT, ISR, IRQ e PIC	Planejado
v0.7	Timer PIT	Adicionar noção de tempo/ticks	Planejado
v0.8	Biblioteca do kernel	Criar funções próprias de string e memória	Planejado
v0.9	Depuração e engenharia reversa	Documentar GDB, objdump, hexdump e análise binária	Planejado
v1.0	Entrega final	Consolidar código, documentação e apresentação	Planejado

5. Plano por etapas

Etapas 1 — v0.1: Kernel mínimo inicializável

Objetivo

Criar a primeira versão funcional do TuringOS, capaz de ser carregada pelo GRUB e executada no QEMU.

Arquivos principais

```
boot/boot.asm
kernel/kernel.c
linker.ld
iso/boot/grub/grub.cfg
Makefile
.gitignore
```

Funcionalidades

- Cabeçalho Multiboot;
- Código Assembly de entrada;
- Configuração de pilha;
- Chamada para C;
- Escrita direta na memória VGA;
- Build automatizado com Makefile;
- Geração de ISO;
- Execução no QEMU.

Critérios de aceitação

A etapa é considerada concluída quando:

```
make
make check
make iso
make run
```

executam corretamente e o QEMU mostra a mensagem do kernel.

Melhorias possíveis dessa etapa

- Corrigir o aviso do linker sobre `.note.GNU-stack`;
- Melhorar o `linker.ld` para separar permissões de segmentos;
- Adicionar `readelf` e `objdump` como comandos de inspeção no Makefile;
- Adicionar um alvo `make info`;
- Adicionar documentação explicando cada parte do boot;
- Adicionar imagem da execução no QEMU em `docs/images`.

Etapa 2 — v0.2: Driver VGA e terminal organizado

Objetivo

Separar o código de vídeo do `kernel.c`, criando um driver VGA reutilizável e um terminal básico.

A versão atual escreve diretamente na VGA dentro do próprio `kernel.c`. Isso funciona, mas ainda é simples. A próxima evolução é organizar o kernel em módulos.

Nova estrutura sugerida

```
include/
  drivers/
    vga.h
  kernel/
    terminal.h
  lib/
    types.h

kernel/
  main.c
  drivers/
    vga.c
  terminal/
    terminal.c
```

Funcionalidades

- Criar `vga.h` e `vga.c`;
- Criar funções para escrever caractere na tela;
- Criar funções para limpar a tela;
- Criar controle de cor;
- Criar controle de linha e coluna;
- Criar função de quebra de linha;
- Criar terminal com `terminal_write`;
- Criar terminal com `terminal_writeln`;
- Atualizar `kernel_main` para usar o terminal;
- Atualizar Makefile para compilar múltiplos arquivos `.c`.

Resultado esperado

Tela inicial mais organizada:

```
TuringOS v0.2
Kernel initialized successfully.

Bootloader : GRUB / Multiboot
Kernel      : C + Assembly x86
Video       : VGA text mode
Status      : OK
```

Critérios de aceitação

A etapa será concluída quando:

- O kernel compilar com múltiplos arquivos C;
- A saída VGA estiver separada em driver;
- O terminal conseguir escrever várias linhas;
- A tela for limpa corretamente;
- O QEMU mostrar a tela inicial formatada;
- O código não depender mais de escrita direta na VGA dentro de `main.c`.

Melhorias possíveis dessa etapa

- Implementar scroll quando passar da linha 25;
- Implementar cursor visual;
- Permitir diferentes cores por mensagem;
- Criar funções `terminal_set_color` e `terminal_reset_color`;
- Criar uma tela de banner do sistema;
- Criar bordas usando caracteres ASCII;
- Criar logs visuais com prefixos `[OK]`, `[INFO]`, `[WARN]`.

Etapa 3 — v0.3: Interface textual inicial

Objetivo

Criar uma interface textual mais apresentável, parecida com uma tela inicial de sistema.

Essa etapa ainda não precisa ter teclado real. O foco é deixar o sistema visualmente mais organizado.

Funcionalidades

- Criar tela de boas-vindas;
- Criar informações do sistema;
- Criar layout textual com bordas;
- Criar área de status;
- Criar prompt visual `turingos>`;
- Criar mensagens de inicialização do kernel.

Exemplo esperado

```
+-----+
|                TuringOS v0.3                |
|          Educational Unix-like Kernel          |
+-----+
| Bootloader : GRUB / Multiboot                |
| Kernel     : C + Assembly x86                |
| Video      : VGA text mode                   |
| Status     : Kernel initialized successfully  |
+-----+

turingos>
```

Critérios de aceitação

- A tela deve ter organização visual clara;
- O terminal deve suportar múltiplas linhas;
- As mensagens devem usar funções do terminal, não acesso direto à VGA;
- A aparência deve ser adequada para apresentação do projeto.

Melhorias possíveis dessa etapa

- Criar tema de cores;
- Criar tela de boot com sequência de inicialização;
- Criar mensagens simulando carregamento de módulos;
- Criar função `kernel_panic`;
- Criar função `print_status`;
- Criar documentação visual da interface.

Etapa 4 — v0.4: Shell básico

Objetivo

Criar uma estrutura inicial de shell com comandos internos.

Inicialmente, o shell pode ser não interativo ou parcialmente simulado. Depois, com o teclado, ele se torna interativo.

Comandos sugeridos

```
help
clear
about
version
meminfo
hardware
reboot
halt
```

Funcionalidades

- Criar `shell.h`;
- Criar `shell.c`;
- Criar função `shell_init`;
- Criar função `shell_execute`;
- Criar comandos internos;
- Criar estrutura de comparação de strings;
- Integrar o shell ao terminal.

Critérios de aceitação

- O kernel deve exibir comandos disponíveis;
- O comando `help` deve listar os comandos;
- O comando `about` deve explicar o projeto;
- O comando `version` deve mostrar a versão atual;
- O comando `clear` deve limpar a tela;
- A lógica do shell deve estar separada do terminal.

Melhorias possíveis dessa etapa

- Criar histórico de comandos;
- Criar autocomplete no futuro;
- Criar parser simples;
- Criar comandos com argumentos;
- Criar comando `banner`;
- Criar comando `uptime` após implementar timer.

Etapa 5 — v0.5: Driver de teclado PS/2

Objetivo

Adicionar entrada de usuário por teclado, tornando o shell realmente interativo.

Conceitos envolvidos

- Porta `0x60`;
- Porta `0x64`;
- Scancodes;
- Conversão de scancode para caractere;
- Leitura de teclado por polling inicialmente.

Funcionalidades

- Criar `keyboard.h`;
- Criar `keyboard.c`;
- Ler scancodes da porta `0x60`;
- Converter letras e números básicos;
- Exibir caracteres digitados no terminal;
- Capturar Enter e Backspace;
- Enviar linha digitada para o shell.

Critérios de aceitação

- O usuário consegue digitar no QEMU;
- O texto digitado aparece na tela;
- O Enter envia o comando;
- O Backspace apaga caracteres;
- O shell executa comandos reais digitados pelo usuário.

Melhorias possíveis dessa etapa

- Suporte a Shift;
- Suporte a Caps Lock;
- Suporte a teclado ABNT2;
- Tratamento de teclas especiais;
- Buffer de teclado;
- Separar scancodes em tabela própria;
- Trocar leitura por polling para leitura por interrupção na etapa seguinte.

Etapa 6 — v0.6: Interrupções, IDT, ISR, IRQ e PIC

Objetivo

Implementar a base de interrupções do kernel.

Essa é uma das etapas mais importantes para transformar o projeto em algo mais avançado, pois aproxima o TuringOS do funcionamento real de um sistema operacional.

Conceitos envolvidos

- IDT — Interrupt Descriptor Table;
- ISR — Interrupt Service Routines;
- IRQ — Hardware Interrupt Requests;
- PIC — Programmable Interrupt Controller;
- Máscaras de interrupção;
- `sti` e `cli`;
- Tratamento de exceções da CPU.

Funcionalidades

- Criar estrutura da IDT;
- Criar funções para instalar entradas na IDT;
- Criar stubs em Assembly para interrupções;
- Criar handlers em C;
- Remapear o PIC;
- Habilitar interrupções;
- Tratar IRQ0 e IRQ1;
- Preparar teclado por interrupção.

Critérios de aceitação

- O kernel inicializa a IDT sem travar;
- Exceções básicas são tratadas;
- O PIC é remapeado;
- O teclado pode funcionar por IRQ1;
- O timer pode funcionar por IRQ0;
- O QEMU continua executando sem triple fault.

Melhorias possíveis dessa etapa

- Criar tela de erro para exceções;
- Criar `kernel_panic`;
- Mostrar registradores em falhas;
- Criar logs de interrupções;
- Criar documentação detalhada da IDT;
- Criar exemplos com GDB inspecionando interrupções.

Etapa 7 — v0.7: Timer PIT

Objetivo

Adicionar uma noção básica de tempo ao kernel usando o Programmable Interval Timer.

Funcionalidades

- Configurar PIT;
- Criar contador de ticks;
- Criar função `timer_init`;

- Criar função `timer_get_ticks`;
- Criar comando `uptime`;
- Integrar timer com IRQ0.

Critérios de aceitação

- O kernel deve contar ticks;
- O comando `uptime` deve exibir o tempo aproximado desde o boot;
- O sistema deve continuar recebendo teclado;
- O timer deve funcionar por interrupção.

Melhorias possíveis dessa etapa

- Criar função `sleep`;
- Criar animação textual simples;
- Criar atualização de relógio na tela;
- Criar benchmark simples;
- Medir tempo de execução de rotinas do kernel.

Etapa 8 — v0.8: Biblioteca básica do kernel

Objetivo

Criar uma pequena biblioteca interna para o kernel, evitando dependência da biblioteca padrão do C.

Como o kernel usa `-ffreestanding` e `-nostdlib`, funções como `strlen`, `strcmp`, `memset` e `memcpy` precisam ser implementadas manualmente.

Funcionalidades

- Criar `string.h`;
- Criar `string.c`;
- Implementar `strlen`;
- Implementar `strcmp`;
- Implementar `strncmp`;
- Implementar `strcpy`;
- Implementar `memset`;
- Implementar `memcpy`;
- Implementar conversão simples de inteiro para string;
- Usar essas funções no terminal e no shell.

Critérios de aceitação

- O shell deve usar `strcmp` próprio;
- O terminal deve usar funções internas;
- O kernel deve continuar compilando sem biblioteca padrão;
- As funções devem ser simples, comentadas e testadas por uso.

Melhorias possíveis dessa etapa

- Criar testes internos simples;
- Criar função `itoa`;
- Criar função `hex_to_string`;
- Criar função `kprintf` simplificada;
- Criar formatação com `%d`, `%x`, `%s`;
- Criar documentação explicando por que o kernel não usa `stdio.h`.

Etapa 9 — v0.9: Engenharia reversa e depuração

Objetivo

Atender explicitamente aos requisitos da disciplina relacionados a depuração, prototipação virtual e engenharia reversa sem código-fonte.

Ferramentas

```
QEMU
GDB
objdump
readelf
hexdump
xxd
ndisasm
grub-file
```

Funcionalidades/documentos

Criar ou atualizar:

```
docs/debugging.md
docs/reverse-engineering.md
docs/build-process.md
docs/binary-analysis.md
```

Atividades de depuração

- Rodar kernel no QEMU;
- Rodar QEMU com `-s -S`;
- Conectar GDB;
- Inspeccionar ponto de entrada;
- Colocar breakpoint em `kernel_main`;
- Inspeccionar registradores;
- Inspeccionar memória VGA em `0xB8000`.

Atividades de engenharia reversa

- Gerar binário do kernel;
- Analisar com `objdump`;
- Ver cabeçalho ELF com `readelf`;
- Ver bytes com `hexdump`;
- Desmontar trechos com `ndisasm`;
- Identificar símbolos;
- Identificar a chamada de `kernel_main`;
- Identificar strings no binário;
- Documentar o que pode ser descoberto sem olhar o código-fonte.

Critérios de aceitação

- Deve existir documentação mostrando comandos e resultados;
- O projeto deve demonstrar análise de binário sem depender do código-fonte;
- A documentação deve explicar o processo de depuração com QEMU/GDB;
- Deve haver evidências para apresentação.

Melhorias possíveis dessa etapa

- Criar script `scripts/debug.sh`;
- Criar script `scripts/analyze-binary.sh`;
- Adicionar alvo `make disasm`;
- Adicionar alvo `make readelf`;
- Adicionar alvo `make hexdump`;
- Adicionar prints da análise na documentação;
- Criar uma seção comparando código C, Assembly gerado e binário final.

Etapa 10 — v1.0: Consolidação e entrega final

Objetivo

Consolidar o projeto para entrega final, garantindo que código, documentação e apresentação estejam consistentes.

Atividades

- Revisar README;
- Revisar documentação técnica;
- Revisar plano de desenvolvimento;
- Atualizar Kanban;
- Fechar issues concluídas;
- Criar tag de versão;
- Testar build do zero;
- Testar execução no QEMU;
- Revisar slides;
- Preparar roteiro de apresentação;
- Criar checklist de requisitos da disciplina.

Critérios de aceitação

O projeto estará pronto para entrega quando:

- O repositório estiver organizado;
- O README explicar o projeto;
- O build funcionar com `make`;
- O QEMU executar o kernel;
- A documentação explicar boot, kernel, VGA, terminal, depuração e engenharia reversa;
- O Kanban mostrar planejamento;
- O histórico Git mostrar evolução;
- A apresentação explicar claramente as decisões técnicas;
- Os requisitos obrigatórios da disciplina estiverem mapeados.

Melhorias possíveis dessa etapa

- Criar release no GitHub;
- Criar tag `v1.0`;
- Criar vídeo curto de demonstração;
- Criar GIF do boot no QEMU;
- Adicionar badges no README;
- Adicionar seção de limitações conhecidas;
- Adicionar seção de trabalhos futuros.

6. Organização do Kanban

Backlog

- Implementar suporte a teclado;
- Implementar interrupções;
- Implementar timer;
- Criar biblioteca de strings;
- Criar shell com comandos;
- Criar documentação de engenharia reversa;
- Criar documentação de depuração;
- Criar apresentação final;
- Criar vídeo ou GIF de demonstração.

A fazer

- Criar driver VGA modular;
- Criar terminal com cursor;
- Atualizar Makefile para múltiplos arquivos;
- Criar tela inicial v0.2;
- Documentar arquitetura do kernel.

Em desenvolvimento

- Funcionalidade da branch atual.

Em teste

- Funcionalidades já implementadas aguardando execução no QEMU.

Concluído

- Estrutura inicial;
 - Kernel mínimo;
 - Boot via GRUB;
 - Escrita em VGA;
 - Makefile inicial;
 - Execução no QEMU.
-

7. Melhorias futuras do TuringOS

Após a entrega principal, o TuringOS pode evoluir para incluir funcionalidades mais avançadas.

Melhorias técnicas futuras

- Paginação de memória;
- Gerenciamento de memória física;
- Alocador dinâmico `kmalloc`;
- Sistema de arquivos em RAM;
- Execução de programas simples;
- Chamadas de sistema reais;
- Separação entre modo kernel e modo usuário;
- Multitarefa cooperativa;
- Multitarefa preemptiva;
- Driver serial;
- Driver de disco;
- Suporte a framebuffer gráfico;
- Interface gráfica experimental;
- Mouse PS/2;
- Loader de módulos;
- Sistema de logs do kernel.

Melhorias de documentação

- Manual do desenvolvedor;
- Manual do usuário;
- Guia de build do zero;
- Guia de depuração;
- Guia de engenharia reversa;
- Diagramas de arquitetura;
- Diagramas de fluxo de boot;
- Diagrama de memória;
- Roteiro de apresentação;
- Registro de decisões técnicas.

Melhorias de engenharia de software

- Scripts de automação;
- Padronização de commits;
- GitHub Actions para build;
- Releases versionadas;
- Tags por versão;
- Issues com milestones;
- Pull requests por funcionalidade;
- Templates de issue e PR;
- Checklist de revisão.

Melhorias acadêmicas

- Explicação detalhada da interface hardware/software;
 - Comparação com Unix;
 - Comparação com Linux;
 - Explicação da diferença entre bootloader e kernel;
 - Explicação do papel do Assembly;
 - Explicação da geração de código de máquina;
 - Demonstração de binário sem código-fonte;
 - Demonstração com QEMU/GDB;
 - Mapeamento explícito dos requisitos da disciplina.
-

8. Riscos do projeto e mitigação

Risco 1 — Escopo grande demais

O desenvolvimento de sistemas operacionais pode crescer rapidamente.

Mitigação:

- Priorizar entregas pequenas;
- Manter v0.2, v0.3 e v0.4 como foco principal;
- Tratar interrupções e timer como avanço, não como dependência da entrega mínima.

Risco 2 — Erros difíceis de depurar

Falhas no kernel podem travar o QEMU sem mensagem clara.

Mitigação:

- Usar QEMU com GDB;
- Criar logs visuais no terminal;
- Testar uma mudança por vez;
- Fazer commits pequenos.

Risco 3 — Problemas com ambiente

Compilação cross-platform pode falhar no Windows.

Mitigação:

- Usar WSL/Linux como ambiente principal;
- Documentar dependências;
- Criar Makefile padronizado;
- Evitar caminhos específicos do Windows.

Risco 4 — Código acoplado demais

Se tudo ficar em `kernel.c`, o projeto parecerá simples e difícil de evoluir.

Mitigação:

- Modularizar o código;
- Criar drivers;
- Criar headers;
- Separar terminal, shell, teclado e biblioteca.

Risco 5 — Falta de evidências para apresentação

Só mostrar código pode não ser suficiente.

Mitigação:

- Guardar prints do QEMU;
- Documentar comandos;
- Mostrar análise com `objdump`;
- Mostrar execução com GDB;
- Manter Kanban atualizado.

9. Estratégia de commits

Sugestão de commits futuros:

```
feat: add VGA text mode driver
feat: add terminal abstraction
feat: add formatted boot screen
feat: add basic shell commands
feat: add kernel string library
feat: add PS/2 keyboard polling
feat: add interrupt descriptor table
feat: add PIC remapping
feat: add PIT timer
```



```
docs: add debugging guide
docs: add reverse engineering guide
docs: update development plan
chore: update Makefile targets
```

10. Próxima ação imediata

A próxima branch recomendada é:

```
feature/vga-terminal
```

Comando:

```
git switch main
git pull origin main
git switch -c feature/vga-terminal
```

Objetivo da branch:

Transformar a saída VGA simples em um driver VGA modular e criar uma camada de terminal reutilizável.

Arquivos esperados:

```
include/drivers/vga.h
include/kernel/terminal.h
kernel/main.c
kernel/drivers/vga.c
kernel/terminal/terminal.c
Makefile
```

Critério de conclusão da branch:

O kernel deve inicializar no QEMU exibindo uma tela formatada usando funções de terminal e driver VGA separados.

11. Conclusão

O TuringOS já possui uma base funcional: o kernel inicializa, executa código C e escreve diretamente na memória VGA. A partir desse ponto, o desenvolvimento deve focar em organização, modularização e interação com hardware.

A evolução ideal é:

```
v0.1 Kernel mínimo
↓
v0.2 Driver VGA + terminal
↓
v0.3 Interface textual
↓
v0.4 Shell
↓
v0.5 Teclado
↓
v0.6 Interrupções
↓
v0.7 Timer
↓
v0.8 Biblioteca interna
↓
v0.9 Debug + engenharia reversa
↓
v1.0 Entrega final
```

Esse plano permite que o projeto cresça de forma controlada, mantendo entregas testáveis e demonstráveis em cada etapa.